



BACHAREL EM CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: DESENVOLVIMENTO WEB 1

TRABALHO FINAL: CATÁLOGO EM BANCO DE DADOS

Evolução Arquitetural de Aplicação Estática em JSON para
Sistema Relacional Dinâmico com PHP e MySQL

Acadêmico: Carlos Gabriel Fontanive

Professor: Prof. Wesley Dos Reis Bezerra

Rio do Sul – SC

07/2026

Conteúdo

1	Introdução e Argumentação do Conteúdo	2
2	Ambiente de Desenvolvimento e Configuração Nativa	2
3	Modelagem e Estrutura do Banco de Dados	4
4	Arquitetura do Sistema e Tecnologias Web	5
5	Apresentação das Telas Desenvolvidas	6
5.1	Página Inicial – Frente do Sistema (index.php)	6
5.2	Listagem do Catálogo – Interna 1 (catalogo.php)	7
5.3	Formulário de Novo Cadastro – Interna 2 (cadastro.php)	8
6	Validação do Fluxo de Dados e Persistência Real	9
7	Conclusão	10

1 INTRODUÇÃO E ARGUMENTAÇÃO DO CONTEÚDO

O presente relatório técnico apresenta o desenvolvimento, a reestruturação arquitetural e a validação do Trabalho Final da disciplina de Desenvolvimento Web 1, ministrada pelo Prof. Wesley Dos Reis Bezerra. O propósito primário desta etapa consistiu em migrar a camada de persistência de um sistema utilitário de catálogo — originalmente construído sobre manipulação de arquivos estáticos no formato JSON — para um ecossistema relacional robusto, utilizando a linguagem PHP estruturada sobre o sistema gerenciador de banco de dados Workbench.

O tema central adotado para a modelagem do catálogo é o gerenciamento de "Hardware, Periféricos e Equipamentos de Tecnologia da Informação", abrangendo computadores de alta performance, periféricos gamers e dispositivos móveis de última geração. A escolha deste domínio justifica-se pela intrínseca necessidade mercadológica do setor de e-commerce e almoxarifados de TI em gerenciar catálogos dinâmicos, onde as especificações técnicas (como capacidade de processamento, memória RAM, litografia e periféricos associados) exigem rápida indexação, atualizações contínuas e leitura estruturada por categorias.

A transição do paradigma de armazenamento em arquivos planos (*flat-files* JSON) para um banco de dados relacional representa um salto evolutivo indispensável para a maturidade do software, solucionando deficiências críticas da modelagem anterior:

- **Eliminação de Concorrência e Corrupção de Dados:** Em sistemas baseados em arquivos JSON, requisições simultâneas de escrita tendem a gerar condições de corrida (*race conditions*), corrompendo a estrutura do arquivo. O banco relacional resolve esse problema através do controle de transações e bloqueio de tabelas (propriedades ACID).
- **Performance e Indexação Estruturada:** A busca por categorias ou ordenação de itens em um arquivo estático exige a leitura integral do arquivo para a memória RAM (deserialização). Com o banco de dados, consultas como `SELECT * ORDER BY id DESC` são resolvidas instantaneamente na camada do motor relacional através de índices B-Tree.
- **Segurança e Escalabilidade Referencial:** O novo modelo prepara a infraestrutura para futuras expansões do catálogo, permitindo relacionamentos normatizados entre tabelas (como vinculação de marcas, fornecedores, controle de estoque quantitativo e histórico de avaliações dos usuários).

2 AMBIENTE DE DESENVOLVIMENTO E CONFIGURAÇÃO NATIVA

Para garantir máximo desempenho, segurança e controle de recursos de hardware, dispensou-se o uso de ambientes empacotados genéricos (como XAMPP ou WAMP). A infraestrutura foi montada diretamente sobre a distribuição Arch Linux, permitindo o gerenciamento modular de dependências, extensões do kernel web e serviços de rede.

A primeira etapa do desenvolvimento consistiu na preparação do interpretador PHP e ativação dos drivers de comunicação com o banco de dados relacional. Conforme ilustrado na Figura 1, foi necessária a verificação e liberação das dependências e extensões de acesso a dados no sistema, especificamente o driver PDO (*PHP Data Objects*) e os módulos de conectividade nativa.

```
$ garrard@nitro-v15-carlos ~/catalogo_limpo
• $ sudo sed -i 's/;extension=pdo_mysql/extension=pdo_mysql/g' /etc/php/php.ini
$ garrard@nitro-v15-carlos ~/catalogo_limpo
• $ sudo sed -i 's/;extension=mysqli/extension=mysqli/g' /etc/php/php.ini
```

Figura 1: Verificação de dependências, extensões do PHP e configuração do ambiente no terminal.

Com as extensões validadas, implementou-se a camada de comunicação com o banco de dados através do script `conexao.php`. Conforme demonstrado na Figura 2, a conexão adota o padrão PDO endereçado via protocolo TCP/IP (127.0.0.1) na porta padrão 3306, estratégia fundamental no Arch Linux para evitar conflitos de permissão de acesso via arquivos de *socket* do sistema operacional.

O código aplica boas práticas de engenharia ao envolver a instância dentro de um bloco de controle de exceções (`try-catch`) e definir o atributo de erro como `PDO::ERRMODE_EXCEPTION`. Essa configuração garante que eventuais falhas na autenticação do usuário `root` ou na seleção do esquema `catalogo_db` interrompam a execução de forma controlada, reportando o log exato da exceção (`PDOException`) sem expor dados sensíveis do servidor.

```
conexao.php
1  <?php
2  $host = '127.0.0.1';
3  $db   = 'catalogo_db';
4  $user = 'root';
5  $pass = 'root';
6
7  try {
8      $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8", $user, $pass);
9      $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
10 } catch (PDOException $e) {
11     die("Erro na conexão: " . $e->getMessage());
12 }
```

Figura 2: Implementação da conexão relacional em `conexao.php` utilizando PDO e tratamento de exceções.

Uma vez validada a camada de persistência e instaladas as bibliotecas de arquitetura via gerenciador de pacotes, o servidor de aplicação foi inicializado utilizando o recurso de servidor web embutido do próprio PHP (`PHP Development Server`). A Figura 3 demonstra a ativação do serviço escutando na porta local 8000 (`http://localhost:8000`), garantindo um ambiente limpo e sem a sobrecarga de módulos Apache desnecessários.

```
$ garrard@nitro-v15-carlos ~/catalogo_limpo
$ php -S localhost:8000
[Mon Jul 13 09:55:24 2026] PHP 8.5.8 Development Server (http://localhost:8000) started
```

Figura 3: Inicialização do servidor web nativo de desenvolvimento do PHP no Arch Linux.

Durante toda a operação da plataforma, o terminal atua como monitor de tráfego HTTP em tempo real. A Figura 4 comprova a comunicação fluida entre o navegador e o back-end, exibindo o processamento das requisições do tipo GET e o retorno de códigos de status [200] OK na renderização das telas do catálogo e dos arquivos de ícone do sistema.

```
$ garrard@nitro-v15-carlos ~/catalogo_limpo
$ php -S localhost:8000
[Mon Jul 13 09:56:12 2026] [::1]:35168 Closing
[Mon Jul 13 09:56:12 2026] [::1]:35172 Accepted
[Mon Jul 13 09:56:12 2026] [::1]:35172 [200]: GET /favicon.ico
[Mon Jul 13 09:56:12 2026] [::1]:35172 Closing
[Mon Jul 13 09:56:17 2026] [::1]:35174 Accepted
[Mon Jul 13 09:56:17 2026] [::1]:35174 [200]: GET /.well-known/appspecific/com.chrome.devtools.json
[Mon Jul 13 09:56:17 2026] [::1]:35174 Closing
[Mon Jul 13 09:56:25 2026] [::1]:36190 Accepted
[Mon Jul 13 09:56:25 2026] [::1]:36190 [200]: GET /catalogo.php
[Mon Jul 13 09:56:25 2026] [::1]:36190 Closing
[Mon Jul 13 09:56:25 2026] [::1]:36202 Accepted
[Mon Jul 13 09:56:25 2026] [::1]:36202 [200]: GET /favicon.ico
[Mon Jul 13 09:56:25 2026] [::1]:36202 Closing
```

Figura 4: Monitoramento de tráfego HTTP e logs de requisição processados pelo servidor PHP.

3 MODELAGEM E ESTRUTURA DO BANCO DE DADOS

Para gerenciar o esquema de dados e realizar a transição do modelo estático, utilizou-se a ferramenta de administração visual **MySQL Workbench**.

A Figura 5 apresenta o script SQL desenvolvido para a estruturação do sistema. O código contempla a declaração de uso do esquema `catalogo_db`, as rotinas de limpeza integral de resíduos de testes anteriores através do comando `TRUNCATE TABLE itens` (garantindo o resete do contador auto-incrementável da chave primária) e a inserção populacional inicial (`INSERT INTO`) com dois registros de carga base para teste de renderização visual: um *Notebook Nitro V15* e um *Mouse Gamer*.

```
USE catalogo_db;

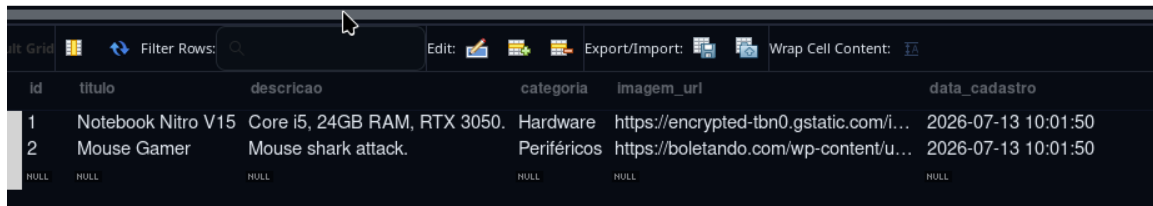
-- Limpa todos os dados e reseta o ID para 1
TRUNCATE TABLE itens;

-- Insere apenas 2 itens limpos para teste
INSERT INTO itens (titulo, descricao, categoria, imagem_url) VALUES
('Notebook Nitro V15', 'Core i5, 24GB RAM, RTX 3050.', 'Hardware', 'https://encrypted-t
('Mouse Gamer', 'Mouse shark attack.', 'Periféricos', 'https://boletando.com/wp-content
```

Figura 5: Script SQL no MySQL Workbench para higienização e inserção dos dados de carga inicial.

Após a execução das rotinas de inicialização da estrutura relacional, procedeu-se à verificação de integridade da tabela. A Figura 6 exibe o retorno da instrução `SELECT * FROM itens`, confirmando a perfeita estruturação das colunas: `id` (Chave Primária), `titulo`, `descricao`, `categoria`, `imagem_url` e o carimbo de data/hora `data_cadastro`, com os dois registros de teste devidamente persistidos em disco.

```
1 • USE catalogo_db;
2 • SELECT * FROM itens;
```



id	titulo	descricao	categoria	imagem_url	data_cadastro
1	Notebook Nitro V15	Core i5, 24GB RAM, RTX 3050.	Hardware	https://encrypted-tbn0.gstatic.com/i...	2026-07-13 10:01:50
2	Mouse Gamer	Mouse shark attack.	Periféricos	https://boletando.com/wp-content/u...	2026-07-13 10:01:50
NULL	NULL	NULL	NULL	NULL	NULL

Figura 6: Verificação na grelha de resultados do Workbench dos registros iniciais da tabela `itens`.

4 ARQUITETURA DO SISTEMA E TECNOLOGIAS WEB

Para atender rigorosamente às especificações técnicas da avaliação, a aplicação foi desenvolvida sob o padrão de separação de responsabilidades, combinando as seguintes tecnologias e frameworks:

- **Back-end em PHP 8.x e Conexão via PDO:** O roteamento e o processamento lógico são geridos em PHP. A comunicação com o Workbench é realizada exclusivamente através da interface PDO (*PHP Data Objects*), aplicando o uso de declarações preparadas (*Prepared Statements*). Esse padrão impede ataques de Injeção de SQL (*SQL Injection*), isolando os dados enviados pelo usuário de dentro do comando SQL interpretado pelo servidor.
- **Motor de Templates Plates PHP:** Para evitar a repetição e a mistura desordenada de lógica de banco com marcação visual, adotou-se o Plates PHP — framework de templates nativo e de altíssimo desempenho recomendado em bibliografia acadêmica. O Plates viabilizou o conceito de *Layout Mestre*, centralizando a estrutura do topo, menu e rodapé em um único arquivo de molde e injetando o conteúdo dinâmico das páginas internas sob demanda, além de tratar o escape automático de saídas para prevenir falhas de segurança do tipo XSS (*Cross-Site Scripting*).
- **HTML5 Semântico:** A estruturação do front-end abandonou o uso genérico de contêineres sem significado, implementando rigorosamente tags semânticas como `<header>`, `<nav>`, `<main>`, `<section>`, `<article>` e `<footer>`. Essa abordagem confere excelência em acessibilidade para leitores de tela e otimização de indexação em motores de busca (SEO).

- **Framework CSS Bootstrap 5:** A padronização visual, a responsividade e o alinhamento em grid foram implementados através da importação via CDN do Bootstrap 5. O framework permitiu criar componentes visuais modernos, botões interativos e cartões de produtos com sombras sutis e tipografia equilibrada.

O reflexo da integração entre o HTML5 Semântico e o framework Bootstrap é evidente na navegação global do sistema. A Figura 7 destaca o cabeçalho superior (<header>) contendo a barra de navegação (<nav>), que proporciona links de acesso instantâneo e intuitivo às três telas fundamentais do catálogo: *Início*, *Catálogo* e *Novo Cadastro*.

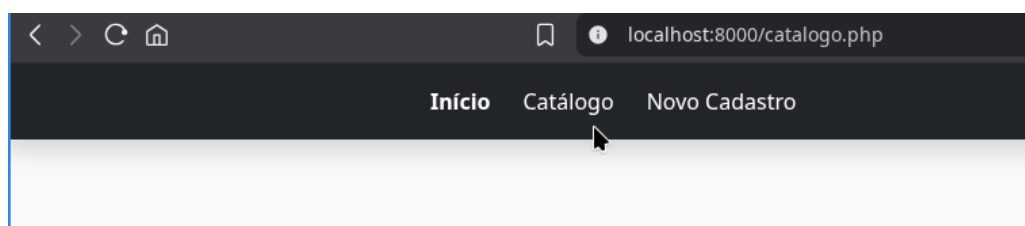


Figura 7: Detalhe da barra de navegação semântica principal estilizada com Bootstrap 5.

5 APRESENTAÇÃO DAS TELAS DESENVOLVIDAS

Em total conformidade com os requisitos de entrega do Trabalho Final, o sistema foi estruturado em três interfaces visuais principais: uma página inicial de apresentação (frente) e duas páginas internas de operação e leitura de dados.

5.1 Página Inicial – Frente do Sistema (index.php)

A primeira tela constitui o ponto de entrada da plataforma. Desenvolvida como uma *Hero Section* centralizada e limpa, a tela acolhe o acadêmico ou cliente, apresenta o título central do catálogo e exibe um resumo da arquitetura do projeto. O botão de chamada para ação (*Call to Action*), estilizado com as classes utilitárias de cores primárias do Bootstrap, direciona o fluxo de navegação imediatamente para a listagem dos produtos no catálogo.

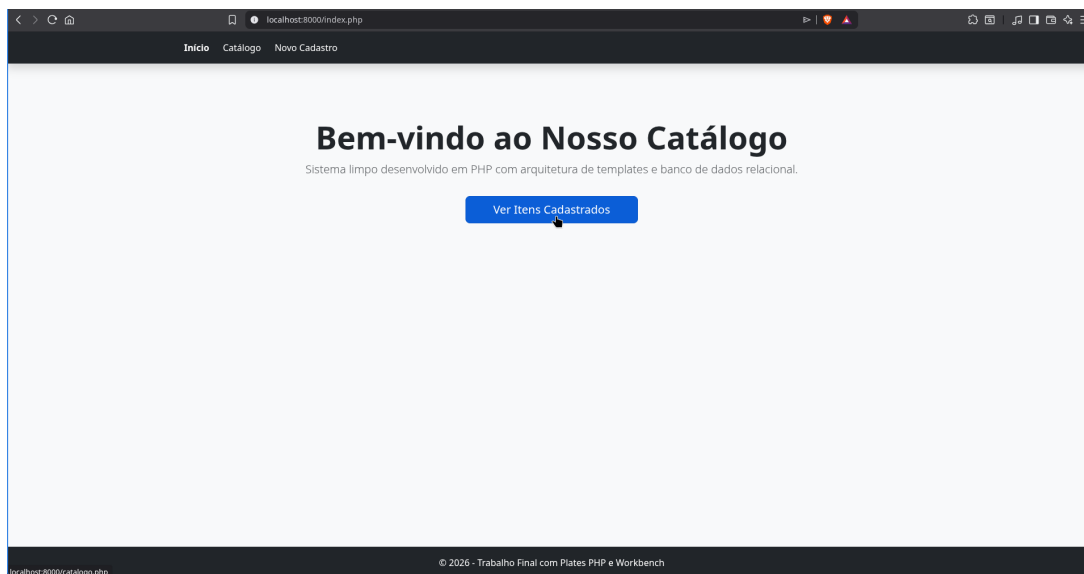


Figura 8: Tela Inicial de Apresentação e Boas-vindas (Frente do Sistema).

5.2 Listagem do Catálogo – Interna 1 (catalogo.php)

A primeira página interna destina-se à leitura dinâmica e renderização gráfica dos itens persistidos no banco de dados. Para cumprir esse papel, a aplicação separa a camada de controle (back-end) da camada de apresentação (front-end).

Conforme ilustrado na Figura 9, o script `catalogo.php` inicia importando o autoloader do Composer e o arquivo de conexão. Em seguida, executa a instrução SQL `SELECT * FROM itens ORDER BY id DESC` através da instância PDO, capturando todos os registros em formato de matriz associativa via parâmetro `PDO::FETCH_ASSOC`. Por fim, instancia o motor do Plates PHP e renderiza o template `lista`, repassando o array de itens como variável para a interface visual.

```

catalogo.php
1  <?php
2  require 'vendor/autoload.php';
3  require 'conexao.php';
4
5  $stmt = $pdo->query("SELECT * FROM itens ORDER BY id DESC");
6  $itens = $stmt->fetchAll(PDO::FETCH_ASSOC);
7
8  $templates = new League\Plates\Engine('templates');
9  echo $templates->render('lista', ['titulo' => 'Nosso Catálogo', 'itens' => $itens]);

```

Figura 9: Implementação do controlador `catalogo.php` realizando a consulta `SELECT` e invocando o template.

A Figura 10 exibe o comportamento da tela ao processar os registros no navegador. Os itens são dispostos de forma responsiva em um sistema de grade (*grid system*) de cartões (*cards*) do Bootstrap 5. Cada cartão apresenta a fotografia do equipamento em proporção padronizada, o título em destaque, a categoria renderizada dentro de uma etiqueta (*badge*) acinzentada e a descrição técnica resumida.

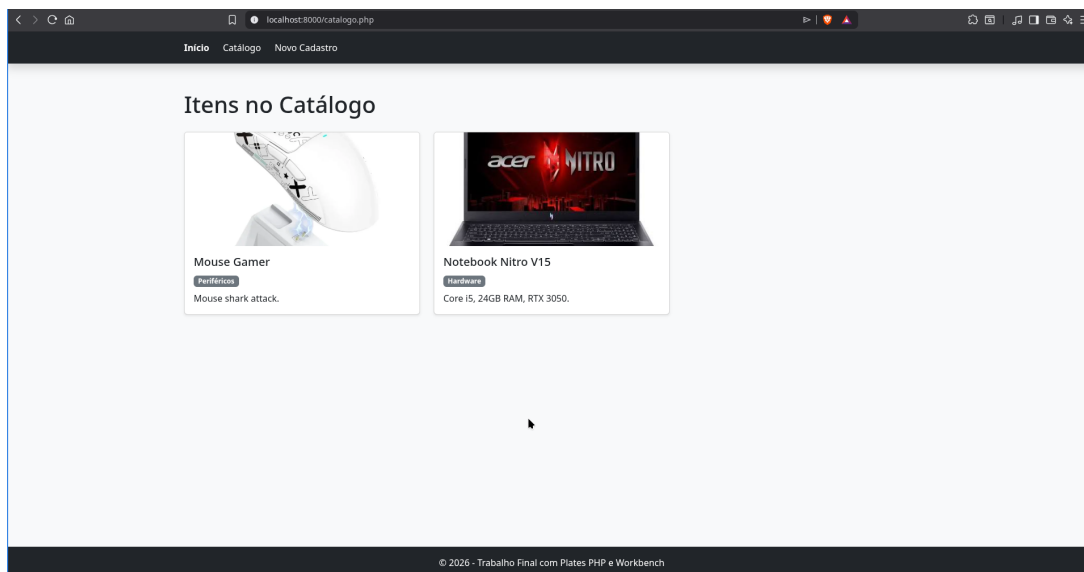


Figura 10: Listagem dinâmica exibindo os produtos de carga base lidos diretamente do Workbench.

5.3 Formulário de Novo Cadastro – Interna 2 (cadastro.php)

A segunda interface interna é dedicada à inserção de novos produtos no catálogo, materializando na prática a substituição dos cadastros em JSON pelos cadastros em banco de dados relacional. A tela é composta por um formulário de fácil usabilidade, estruturado no interior de uma tag semântica `<section>` com largura máxima controlada e fundo estilizado em cartão limpo.

A Figura 11 ilustra o exato momento de utilização da interface, onde os dados de um novo smartphone de alta tecnologia — o *iPhone 14* — são inseridos nos respectivos campos de entrada de texto, categoria, URL de imagem de demonstração e área de descrição técnica.

Figura 11: Formulário HTML semântico sendo preenchido para o cadastro dinâmico do *iPhone 14*.

O ciclo de operação de escrita no back-end ocorre de forma segura e automatizada:

1. O usuário submete os campos do formulário visual acionando o botão de confirmação verde (*Salvar no Banco de Dados*).
2. O navegador empacota as propriedades e as transmite de forma oculta ao controlador via método HTTP POST.
3. O script de recepção intercepta a requisição, sanitiza os valores recebidos e monta a instrução de inserção parametrizada via PDO: `INSERT INTO itens (titulo, descricao, categoria, imagem_url) VALUES (:t, :d, :c, :i)`.
4. Confirmada a gravação física no motor Workbench, o sistema executa um redirecionamento de cabeçalho (`header("Location: catalogo.php")`), conduzindo o usuário de volta à tela de listagem para verificação imediata do resultado.

6 VALIDAÇÃO DO FLUXO DE DADOS E PERSISTÊNCIA REAL

Para comprovar cientificamente que a aplicação executou o ciclo completo de migração relacional — desde o envio dos dados no front-end até a leitura e persistência física em disco no back-end —, foram realizadas duas checagens de verificação pós-cadastro do produto de teste (*iPhone 14*).

A primeira constatação ocorre na própria camada de apresentação. A Figura 12 ilustra o cartão individual do *iPhone 14* renderizado de forma totalmente dinâmica na interface do catálogo, exibindo a imagem capturada da web, a categoria *celular* em destaque e a especificação técnica gravada na descrição. Esse resultado comprova que o template do Plates PHP interpretou com sucesso o novo registro retornado pela instrução de consulta ao banco de dados.

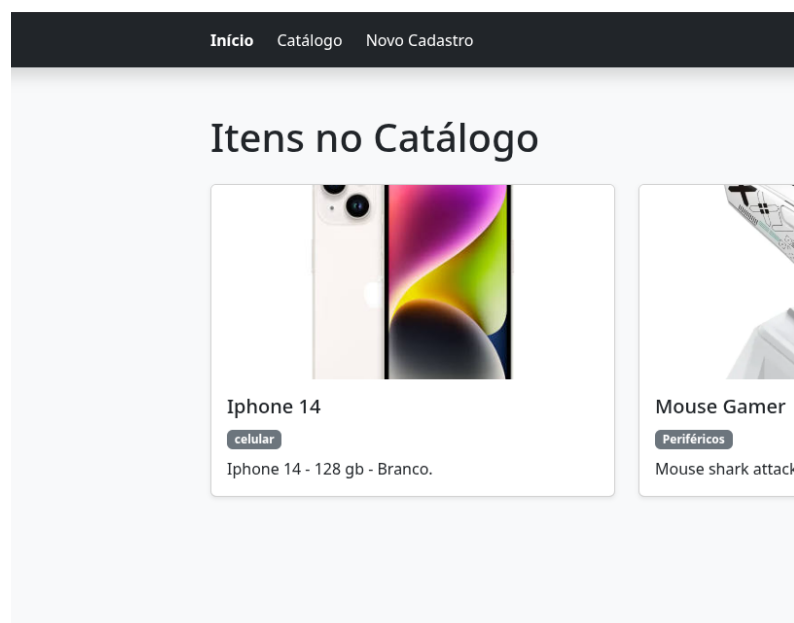
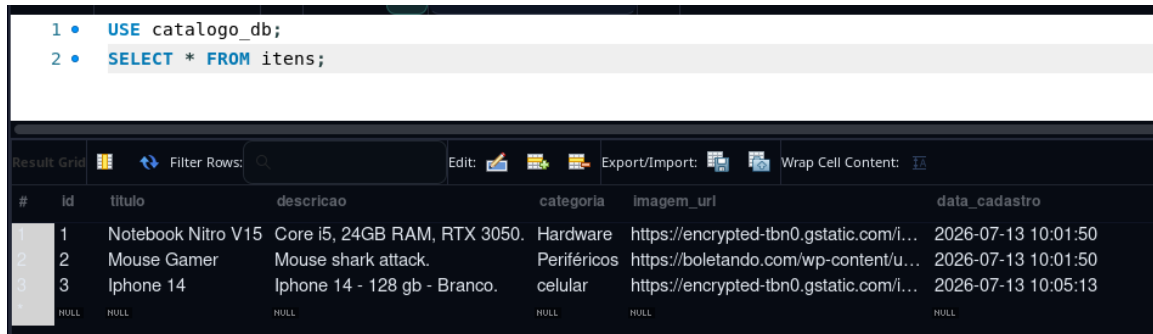


Figura 12: Cartão dinâmico do produto *iPhone 14* renderizado em tempo real na tela do catálogo.

A segunda e definitiva constatação foi realizada na camada profunda de armazenamento. A Figura 13 apresenta a consulta de validação final executada diretamente na interface de administração do MySQL Workbench. O resultado exibe a tabela `itens` devidamente populada com os três registros, comprovando a gravação do *iPhone 14* assumindo o identificador auto-incrementável `id = 3`, validando a integridade referencial do sistema relacional.



The screenshot shows the MySQL Workbench interface. At the top, a SQL editor contains two lines of code: `1 • USE catalogo_db;` and `2 • SELECT * FROM itens;`. Below the editor, the 'Result Grid' tab is active, displaying a table with 7 columns: `#`, `id`, `titulo`, `descricao`, `categoria`, `imagem_url`, and `data_cadastro`. The table contains three rows of data, with the third row highlighted. The third row shows `id = 3`, `titulo = Iphone 14`, `descricao = Iphone 14 - 128 gb - Branco.`, `categoria = celular`, `imagem_url = https://encrypted-tbn0.gstatic.com/i...`, and `data_cadastro = 2026-07-13 10:05:13`. The first two rows show `id = 1` and `id = 2` with their respective product details.

#	id	titulo	descricao	categoria	imagem_url	data_cadastro
1	1	Notebook Nitro V15	Core i5, 24GB RAM, RTX 3050.	Hardware	https://encrypted-tbn0.gstatic.com/i...	2026-07-13 10:01:50
2	2	Mouse Gamer	Mouse shark attack.	Periféricos	https://boletando.com/wp-content/u...	2026-07-13 10:01:50
3	3	Iphone 14	Iphone 14 - 128 gb - Branco.	celular	https://encrypted-tbn0.gstatic.com/i...	2026-07-13 10:05:13

Figura 13: Grelha de resultados no MySQL Workbench comprovando a persistência física do *iPhone 14* (ID 3).

7 CONCLUSÃO

O desenvolvimento deste Trabalho Final atingiu com êxito e rigor técnico a totalidade dos objetivos propostos para a avaliação de encerramento do módulo de Banco de Dados. A substituição bem-sucedida do mecanismo estático de arquivos JSON em favor de uma arquitetura relacional nativa conectada via PDO ao Workbench elevou o sistema ao patamar de uma aplicação da web dinâmica, altamente segura, responsiva e profissional.

A sinergia entre o ambiente Linux modular, a adoção de boas práticas de engenharia de software — expressas pela arquitetura de templates com Plates PHP e o respeito irrestrito à semântica do HTML5 e à elegância visual do Bootstrap 5 — comprova que é plenamente viável construir sistemas web limpos, otimizados e organizados sem a necessidade de recorrer a frameworks monolíticos excessivamente pesados. O trabalho consolida o domínio de todas as etapas do desenvolvimento de software, desde a administração do servidor de banco de dados até a entrega de uma experiência visual interativa ao usuário final.